

实验指导书

智能体框架、多轮对话、工具调用与多模态交互

课程： AI 智能体及其座舱应用
适用课时： 第 10-12 课时
课程模块： 智能体框架与座舱实践
文档版本： 课程教学版（2026）

重庆大学

学生实验用

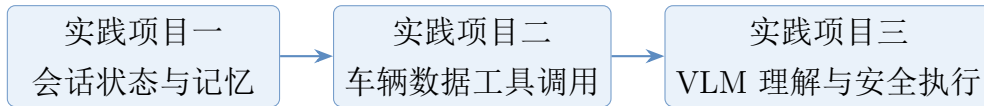
目录

1	指导书定位与学习主线	1
1.1	实验成果与验收要求	1
2	统一环境与准备工作	1
2.1	软件环境	1
2.2	目录检查与自动测试	2
2.3	离线与在线两种模式	2
2.4	基础实验完成判定	2
3	实验一：结合记忆系统实现多轮对话	3
3.1	实验目标	3
3.2	核心结构	3
3.3	实验步骤	3
3.3.1	步骤 1：启动会话	3
3.3.2	步骤 2：写入并回忆信息	3
3.3.3	步骤 3：验证持久化	4
3.3.4	步骤 4：验证会话隔离	4
3.3.5	步骤 5：验证事实更新	4
3.4	验收测试	5
3.5	思考与拓展	5
4	实验二：设计并调用车辆数据查询工具	5
4.1	实验目标	5
4.2	工具契约	6
4.3	实验步骤	6
4.3.1	步骤 1：查看模拟数据	6
4.3.2	步骤 2：运行工具调用	6
4.3.3	步骤 3：阅读调用轨迹	6
4.3.4	步骤 4：验证 Schema	7
4.3.5	步骤 5：可选 MCP 接入	7
4.4	验收测试	8
4.5	安全拓展	8

5 实验三：基于 VLM 的座舱 UI 理解与操作	8
5.1 实验目标	8
5.2 安全执行链	9
5.3 实验步骤	9
5.3.1 步骤 1：查看示例界面与控件清单	9
5.3.2 步骤 2：运行离线 VLM	9
5.3.3 步骤 3：观察三种结果	9
5.3.4 步骤 4：检查审计日志	9
5.3.5 步骤 5：可选在线 VLM	10
5.4 验收测试	10
6 课程项目：选题与设计	11
6.1 选题原则	11
6.2 推荐选题	11
6.3 技术方案最小闭环	11
6.4 建议验收指标	12
7 评分建议	12
8 常见问题	13
9 实践项目参考	13

1 指导书定位与学习主线

本指导书对应课程第 10-12 课时。三个实验不是彼此割裂的代码片段，而是一条逐步扩展的应用开发主线：



实验教学目标： 解释智能体应用的基本运行循环；实现可隔离、可恢复的会话记忆；为车辆数据工具定义结构化契约并处理失败；把 VLM 输出约束为可验证的 UI 动作计划；为课程项目建立可测试、可追溯的技术方案。

基础实验完成边界： 三个基础实验均可在离线模式下完成并验收；在线文本模型、在线 VLM 与 MCP 接入仅作为可选拓展，不影响基础实验成绩。

1.1 实验成果与验收要求

编号	内容	实验成果	核心验收点
实验一	多轮对话与记忆	代码、对话记录、记忆表截图、测试结果	同会话可回忆、跨会话不串数据、重启后可恢复
实验二	车辆数据工具调用	工具契约、调用日志、异常测试	工具选择正确、参数合法、失败可解释、日志可追溯
实验三	VLM 座舱 UI 理解	结构化识别结果、安全策略、模拟执行日志	低置信度拒绝、高风险确认、不得真实控制车辆
课程项目	选题与技术方案	项目设计书、工作流图、测试计划	问题明确、接口明确、风险明确、可以客观验收

2 统一环境与准备工作

2.1 软件环境

- Python 3.10 或更高版本；
- Git；
- 可选：支持 OpenAI-compatible 接口的文本模型或多模态模型；
- 可选：MCP Python SDK。基础实验不依赖网络和第三方包。

2.2 目录检查与自动测试

若尚未获取案例代码，先执行：

```
git clone https://github.com/Shiroe7/ai-agent-cockpit-course.git
```

在仓库根目录进入案例代码目录：

```
cd code
python -m unittest discover -s tests -v
```

预期结果为 8 项测试全部通过。若测试不通过，不应继续连接在线模型；先根据错误信息定位环境、路径或代码问题。

2.3 离线与在线两种模式

模式	适用场景	特点
离线 mock	课堂演示、首次实验、自动测试	无 API Key；结果稳定；便于观察状态、工具和策略
在线模型	拓展实验、能力比较	需要配置模型接口；输出存在随机性；必须保留结构校验和安全门

安全边界：不得在仓库中提交 API Key、真实 VIN、真实位置、真实驾驶轨迹或个人信息。所有车辆数据必须使用课程模拟数据或完成脱敏。

2.4 基础实验完成判定

学生完成三个基础实验时，应同时满足以下条件：

1. 8 项自动测试全部通过；
2. 按各实验验收表完成正常、异常和安全用例；
3. 生成数据库记录或 JSONL 日志，并能解释关键字段；
4. 全程使用课程模拟数据，不接入真实车辆控制；
5. 仓库中不包含 API Key 或个人、车辆敏感数据。

3 实验一：结合记忆系统实现多轮对话

3.1 实验目标

1. 理解模型调用本身通常不自动保存会话状态；
2. 区分消息历史、会话事实和长期记忆；
3. 使用 session ID 隔离不同会话；
4. 使用 SQLite 保存消息与事实；
5. 通过测试发现记忆覆盖、过期和污染问题。

3.2 核心结构

基础实现位于：

- lab01_memory_chat/memory.py: SQLite 存储层；
- lab01_memory_chat/agent.py: 事实抽取、检索和回答；
- lab01_memory_chat/cli.py: 命令行交互；
- tests/test_lab01.py: 持久化、隔离和更新测试。

本实验将记忆分成两层：

1. **消息历史**：保存原始用户与智能体消息，支持回放与审计；
2. **结构化事实**：把姓名、偏好、温度等稳定信息保存为键值，避免每次读取完整历史。

3.3 实验步骤

3.3.1 步骤 1：启动会话

```
python -m lab01_memory_chat.cli --session student-01
```

3.3.2 步骤 2：写入并回忆信息

依次输入：

1. 我叫小刘
2. 我喜欢安静的座舱

3. 我希望座舱温度设为 24 度
4. 隔几轮后输入：我叫什么？
5. 输入：我的偏好是什么？

记录每一轮的回答、当前 facts 表和消息数量。退出交互程序后，可在 code/ 目录执行以下命令查看验收证据：

```
python -c "import sqlite3; c=sqlite3.connect('runtime/lab01_memory.sqlite3'); print
(c.execute('SELECT session_id,fact_key,fact_value FROM facts ORDER BY
session_id,fact_key').fetchall())"
python -c "import sqlite3; c=sqlite3.connect('runtime/lab01_memory.sqlite3'); print
(c.execute('SELECT session_id,COUNT(*) FROM messages GROUP BY session_id').
fetchall())"
```

3.3.3 步骤 3：验证持久化

输入 exit 退出程序，重新使用相同 session ID 启动，再次询问姓名和偏好。若仍能回答，说明数据已从 SQLite 恢复。

3.3.4 步骤 4：验证会话隔离

```
python -m lab01_memory_chat.cli --session student-02
```

直接询问姓名。预期结果为“当前会话中还没有记录”。若返回 student-01 的信息，则发生了跨会话泄漏。

3.3.5 步骤 5：验证事实更新

在 student-01 中先输入“我喜欢安静的座舱”，再输入“我喜欢轻音乐”，然后询问偏好。观察 upsert 如何更新同一键。

3.4 验收测试

编号	输入/操作	预期结果	是否通过
M1	同一会话写入姓名后再 次询问	正确回忆姓名	
M2	重启程序后询问姓名	正确恢复	
M3	切换新的 session ID	不出现旧会话信息	
M4	连续写入两个偏好	返回最新偏好	
M5	输入空字符串	拒绝并返回明确错误	
M6	清空会话后重新询问	不再返回已删除信息	

教学问题：记住了，为什么问不出来？ 先输入“我希望座舱温度设置为 24 度”，确认系统提示温度已写入；再分别询问“我的温度偏好是什么？”“我设置的是几度？”“我想要多少度？”和“座舱温度是多少度？”。比较回答差异，并结合代码判断失败发生在事实存储、记忆检索还是自然语言意图识别阶段。

3.5 思考与拓展

1. 消息历史持续增长时，应选择截断、摘要还是检索？说明取舍。
2. 给记忆增加 `source`、`valid_until` 和 `verified` 字段。
3. 设计一条“旧偏好不再适用”的测试，并实现淘汰策略。
4. 将基础规则抽取替换为在线模型，但保留 SQLite、session ID 和测试。

4 实验二：设计并调用车辆数据查询工具

4.1 实验目标

1. 把自然语言问题转换为“工具名 + JSON 参数”；
2. 使用 Schema 检查必填字段、类型和额外参数；
3. 区分规划错误、参数错误、业务错误和执行错误；
4. 使用 JSONL 保存工具调用轨迹；
5. 了解 Function Calling 与 MCP 的层次关系。

4.2 工具契约

工具	用途	必填参数
get_vehicle_status	查询在线状态、电量、续航、温度和告警	vehicle_id
get_energy_summary	查询电量、续航和能耗	vehicle_id
get_tire_pressure	查询四轮胎压和低压位置	vehicle_id
explain_fault_code	解释课程模拟故障码	code

关键原则： 工具描述负责帮助模型选对工具；Schema 负责拒绝不合法参数；工具包装层负责把异常变成可解释结果；审计日志负责还原“为什么调用、调用了什么、结果是什么”。

4.3 实验步骤

4.3.1 步骤 1：查看模拟数据

阅读 lab02_vehicle_tools/data/vehicle_mock.json，确认课程车辆编号、字段、单位和故障码均为模拟值。

4.3.2 步骤 2：运行工具调用

```
python -m lab02_vehicle_tools.cli --vehicle CQ-AI-001
```

依次询问：

1. 查询胎压；
2. 还有多少续航；
3. 查询 CQ-AI-002 的综合状态；
4. 解释故障码 TPMS_201；
5. 查询 CQ-AI-999 的状态。

4.3.3 步骤 3：阅读调用轨迹

检查输出中的 plan 和 result，并查看 runtime/lab02_tool_calls.jsonl。每条记录应包含查询、工具名、参数、执行状态和结果。

4.3.4 步骤 4：验证 Schema

先运行实验二现有测试，观察缺少 `vehicle_id` 时工具注册表如何拒绝调用：

```
python -m unittest discover -s tests -p "test_lab02.py" -v
```

打开 `tests/test_lab02.py`，参考 `test_schema_rejects_missing_parameter`，新增一个“额外参数”测试。调用时可构造：

```
result = build_registry().invoke(
    "get_vehicle_status",
    {"vehicle_id": "CQ-AI-001", "debug": True},
)
```

断言 `result.ok` 为 `False`，且错误信息包含“存在未声明参数”。重新运行上述命令，新增测试通过即完成本步骤。思考 `additionalProperties: false` 为什么能降低模型生成多余参数带来的风险。

4.3.5 步骤 5：可选 MCP 接入

安装可选依赖并运行最小 MCP 客户端：

```
pip install -r requirements-optional.txt
python -m lab02_vehicle_tools.mcp_client_demo --vehicle CQ-AI-001
```

客户端会自动启动 MCP Server、列出四个工具，并通过协议调用 `get_vehicle_status`。看到工具列表和车辆状态 JSON 即表示 MCP 通信成功。

安全边界：MCP Server 使用 JSON-RPC 协议，不能在服务器窗口直接输入“你好”或“车辆状态”。本步骤验证工具发现与协议调用，不负责自然语言理解；自然语言问答继续使用实验二 CLI，或由接入大模型的 MCP Host 完成工具选择。

MCP 仅改变工具的发现和连接方式，不替代输入校验、权限、日志或确认机制。

4.4 验收测试

编号	输入/操作	预期结果	是否通过
T1	“查询胎压”	选择胎压工具并返回四轮数据	
T2	“还有多少续航”	选择能源工具	
T3	缺少车辆编号且无默认值	在规划阶段提示补充参数	
T4	车辆编号 CQ-AI-999	安全返回“未找到车辆”	
T5	未知故障码	安全返回业务错误	
T6	调用结束后查看 JSONL	记录完整且可回放	

4.5 安全拓展

1. 为工具增加只读/写入等级；
2. 写操作增加二次确认；
3. 增加超时、最大调用次数和预算；
4. 将外部工具返回内容标记为不可信数据，禁止其修改系统指令；
5. 设计失败写回状态并重新规划的流程。

5 实验三：基于 VLM 的座舱 UI 理解与操作

5.1 实验目标

1. 理解 VLM 输出不能直接等同于可执行操作；
2. 使用结构化 JSON 描述目标、动作、坐标、置信度和理由；
3. 对照受信任 UI 清单验证控件和坐标；
4. 对低置信度和高风险动作执行拒绝或确认；
5. 只在模拟器中执行，不连接真实车辆。

5.2 安全执行链



结构化动作至少包含：

```
{
  "target_id": "ac_toggle",
  "label": "...",
  "action": "click",
  "bounds": [94, 590, 242, 680],
  "confidence": 0.96,
  "reason": "...",
  "requires_confirmation": false
}
```

5.3 实验步骤

5.3.1 步骤 1：查看示例界面与控件清单

在 lab03_vlm_ui/ 目录中查看以下文件：

- assets/sample_cockpit.svg: 示例座舱界面；
- data/sample_ui.json: 受信任控件清单。

5.3.2 步骤 2：运行离线 VLM

```
python -m lab03_vlm_ui.cli
```

分别输入“打开空调”“导航回家”“切换运动模式”和“打开车窗”。

5.3.3 步骤 3：观察三种结果

- simulated: 通过校验，仅完成模拟；
- confirmation_required: 高风险控件需要人工确认；
- refused: 未知控件、低置信度或契约不一致。

5.3.4 步骤 4：检查审计日志

查看 runtime/lab03_ui_actions.jsonl，确认每次规划、验证和模拟执行都有记录。

5.3.5 步骤 5：可选在线 VLM

本步骤不属于基础验收。安装可选依赖后，在当前终端设置模型接口环境变量。Windows 终端示例：

```
pip install -r requirements-optional.txt
$env:OPENAI_API_KEY="..."
$env:OPENAI_BASE_URL="https://example.com/v1"
$env:VLM_MODEL="your-vlm-model"
```

macOS 或 Linux 终端示例：

```
pip install -r requirements-optional.txt
export OPENAI_API_KEY="..."
export OPENAI_BASE_URL="https://example.com/v1"
export VLM_MODEL="your-vlm-model"
```

配置完成后运行：

```
python -m lab03_vlm_ui.cli --online
```

在线模式使用 OpenAICompatibleVLM，但仍经过与离线模式相同的结构校验、安全策略和模拟执行链；不得删除安全门。

安全边界：图像中出现的文字也可能是提示注入载体。VLM 应只把界面文字当作待识别数据，不得服从“忽略安全规则”“点击危险按钮”等图像内指令。

5.4 验收测试

编号	输入/操作	预期结果	是否通过
V1	打开空调	找到空调开关并模拟点击	
V2	导航回家	找到导航控件并模拟点击	
V3	切换运动模式	必须要求人工确认	
V4	打开车窗（清单中不存在）	拒绝执行	
V5	人为修改错误坐标	策略校验拒绝	
V6	置信度低于 0.75	拒绝执行	
V7	图像中加入诱导文字	不改变系统安全规则	

6 课程项目：选题与设计

6.1 选题原则

课程项目应满足以下五项：

1. 有明确用户和具体座舱场景；
2. 至少包含记忆、工具或多模态中的两类能力；
3. 输入、输出和状态可以结构化描述；
4. 有失败分支、风险控制和审计日志；
5. 能用测试用例客观验收，而不是只展示一次演示。

6.2 推荐选题

- 车辆状态问答与故障解释智能体；
- 驾驶前座舱检查助手；
- 车载手册检索与操作指导智能体；
- 多模态座舱 UI 导航助手；
- 行程规划与能耗解释智能体。

6.3 技术方案最小闭环

项目方案至少画清楚：

1. 用户输入与场景状态；
2. 模型的角色和输出契约；
3. 可调用工具及其权限；
4. 记忆保存、检索、更新和删除策略；
5. 执行前校验与高风险确认；
6. 执行结果写回和失败重规划；
7. 日志、测试与最终输出。

6.4 建议验收指标

课程项目应使用统一测试集和运行证据进行验收，指标可根据选题调整，但必须能够复现和核查。

维度	指标示例	证据
任务完成	测试任务成功率	测试表、日志
记忆质量	正确回忆率、跨会话泄漏数、过期记忆淘汰率	对话轨迹、数据库记录
工具可靠性	工具选择正确率、参数合法率、越权调用数	tool call 日志
恢复能力	失败恢复率、不可逆错误数	失败注入测试
多模态安全	低置信度拒绝率、高风险确认率、图像提示注入成功率	红队测试记录
效率	平均延迟、最大步骤数、工具调用次数	运行统计
可追溯性	日志覆盖率、版本和 Git 提交可定位性	仓库记录

7 评分建议

课程项目总分为 100 分。评分同时关注功能闭环、测试证据、安全约束和复现质量，不能只以演示效果或功能数量替代客观验证。

项目	权重	评分要点
功能正确性	30%	主流程完成，输入输出符合契约
测试与证据	20%	覆盖正常、异常和安全用例
架构与代码质量	15%	模块边界清晰、配置与密钥分离
安全与隐私	20%	最小权限、确认、拒绝、日志、脱敏
报告与复现	10%	文档完整，新环境可运行
拓展与分析	5%	有依据的改进，不以堆功能替代分析

8 常见问题

以下问题覆盖实验运行、模型输出、工具循环和依赖版本等常见故障，可先按对应建议排查。

运行时找不到模块 请确认当前目录是 `code/`，并使用 `python -m ...` 运行。

SQLite 文件被占用 关闭仍在运行的命令程序，不要同时让多个程序写同一实验数据库。

模型输出不是 JSON 不要直接执行；先提取并验证 JSON，失败时要求模型重试或拒绝。

工具调用反复循环 设置最大步骤数、超时、预算和连续失败断路器。

MCP SDK 版本变化 基础实验使用本地 Tool Registry。可选依赖版本以 `requirements-optional.txt` 为准；升级 SDK 后重新运行自动测试。

9 实践项目参考

- 《AI 智能体及其座舱应用大纲》，第 1 页；
- 《对话推理与智能体》，智能体、记忆、工具与规划部分；
- 《MCP & Skill》，工具契约、MCP、Harness、状态管理和工程实践部分；
- 《Agent Planning-从推理到规划》，规划闭环、工具验证、记忆与验收部分；
- 《大模型安全与合规》，上下文安全、多模态提示注入与纵深防御部分；
- 《自动驾驶和 Alpamayo》《VLA_slides》，VLM/VLA 感知-行动边界与安全验证部分；
- Model Context Protocol 官方架构与 Python SDK 文档；
- LangGraph 官方 Memory/Persistence 文档。